

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import sqlite3
from scipy.stats import ttest_ind
import scipy.stats as stats
warnings.filterwarnings("ignore")

```

Loading the datasets

```

# creating database connection
conn = sqlite3.connect("inventory.db")

df = pd.read_sql_query("SELECT * FROM vendor_sales_summary", conn)
df.head()

```

	VendorNumber	VendorName	Brand
Description \			
0	1128	BROWN-FORMAN CORP	1233 Jack Daniels No 7
Black			
1	4425	MARTIGNETTI COMPANIES	3405 Tito's Handmade
Vodka			
2	17035	PERNOD RICARD USA	8068 Absolut 80
Proof			
3	3960	DIAGEO NORTH AMERICA INC	4261 Capt Morgan Spiced
Rum			
4	3960	DIAGEO NORTH AMERICA INC	3545 Ketel One
Vodka			

	PurchasePrice	ActualPrice	Volume	TotalPurchaseQuantity \
0	26.27	36.99	1750.0	145080
1	23.19	28.99	1750.0	164038
2	18.24	24.99	1750.0	187407
3	16.17	22.99	1750.0	201682
4	21.89	29.99	1750.0	138109

	TotalPurchaseDollars	TotalSalesQuantity	TotalSalesDollars \
0	3811251.60	142049.0	5101919.51
1	3804041.22	160247.0	4819073.49
2	3418303.68	187140.0	4538120.60
3	3261197.94	200412.0	4475972.88
4	3023206.01	135838.0	4223107.62

	TotalSalesPrice	TotalExciseTax	FreightCost	GrossProfit
ProfitMargin \				
0	672819.31	260999.20	68601.68	1290667.91

```

25.297693
1      561512.37      294438.66      144929.24      1015032.27
21.062810
2      461140.15      343854.07      123780.22      1119816.92
24.675786
3      420050.01      368242.80      257032.07      1214774.94
27.139908
4      545778.28      249587.83      257032.07      1199901.61
28.412764

      StockTurnover  SalesToPurchaseRatio
0      0.979108      1.338647
1      0.976890      1.266830
2      0.998575      1.327594
3      0.993703      1.372493
4      0.983556      1.396897

df.to_csv("vendor_sales_summary.csv", index=False)

```

Exploratory Data Analysis

```

# summary statistics
df.describe().T

```

	count	mean	std	min
\				
VendorNumber	10692.0	1.065065e+04	18753.519148	2.00
Brand	10692.0	1.803923e+04	12662.187074	58.00
PurchasePrice	10692.0	2.438530e+01	109.269375	0.36
ActualPrice	10692.0	3.564367e+01	148.246016	0.49
Volume	10692.0	8.473605e+02	664.309212	50.00
TotalPurchaseQuantity	10692.0	3.140887e+03	11095.086769	1.00
TotalPurchaseDollars	10692.0	3.010669e+04	123067.799627	0.71
TotalSalesQuantity	10692.0	3.077482e+03	10952.851391	0.00
TotalSalesDollars	10692.0	4.223907e+04	167655.265984	0.00
TotalSalesPrice	10692.0	1.879378e+04	44952.773386	0.00
TotalExciseTax	10692.0	1.774226e+03	10975.582240	0.00
FreightCost	10692.0	6.143376e+04	60938.458032	0.09

GrossProfit	10692.0	1.213238e+04	46224.337964	-52002.78
ProfitMargin	10692.0	-inf	NaN	-inf
StockTurnover	10692.0	1.706793e+00	6.020460	0.00
SalesToPurchaseRatio	10692.0	2.504390e+00	8.459067	0.00

	25%	50%	75%
max			
VendorNumber	3951.000000	7153.000000	9552.000000
2.013590e+05			
Brand	5793.500000	18761.500000	25514.250000
9.063100e+04			
PurchasePrice	6.840000	10.455000	19.482500
5.681810e+03			
ActualPrice	10.990000	15.990000	28.990000
7.499990e+03			
Volume	750.000000	750.000000	750.000000
2.000000e+04			
TotalPurchaseQuantity	36.000000	262.000000	1975.750000
3.376600e+05			
TotalPurchaseDollars	453.457500	3655.465000	20738.245000
3.811252e+06			
TotalSalesQuantity	33.000000	261.000000	1929.250000
3.349390e+05			
TotalSalesDollars	729.220000	5298.045000	28396.915000
5.101920e+06			
TotalSalesPrice	289.710000	2857.800000	16059.562500
6.728193e+05			
TotalExciseTax	4.800000	46.570000	418.650000
3.682428e+05			
FreightCost	14069.870000	50293.620000	79528.990000
2.570321e+05			
GrossProfit	52.920000	1399.640000	8660.200000
1.290668e+06			
ProfitMargin	13.324515	30.405457	39.956135
9.971666e+01			
StockTurnover	0.807229	0.981529	1.039342
2.745000e+02			
SalesToPurchaseRatio	1.153729	1.436894	1.665449
3.529286e+02			

Distribution plots for numerical columns

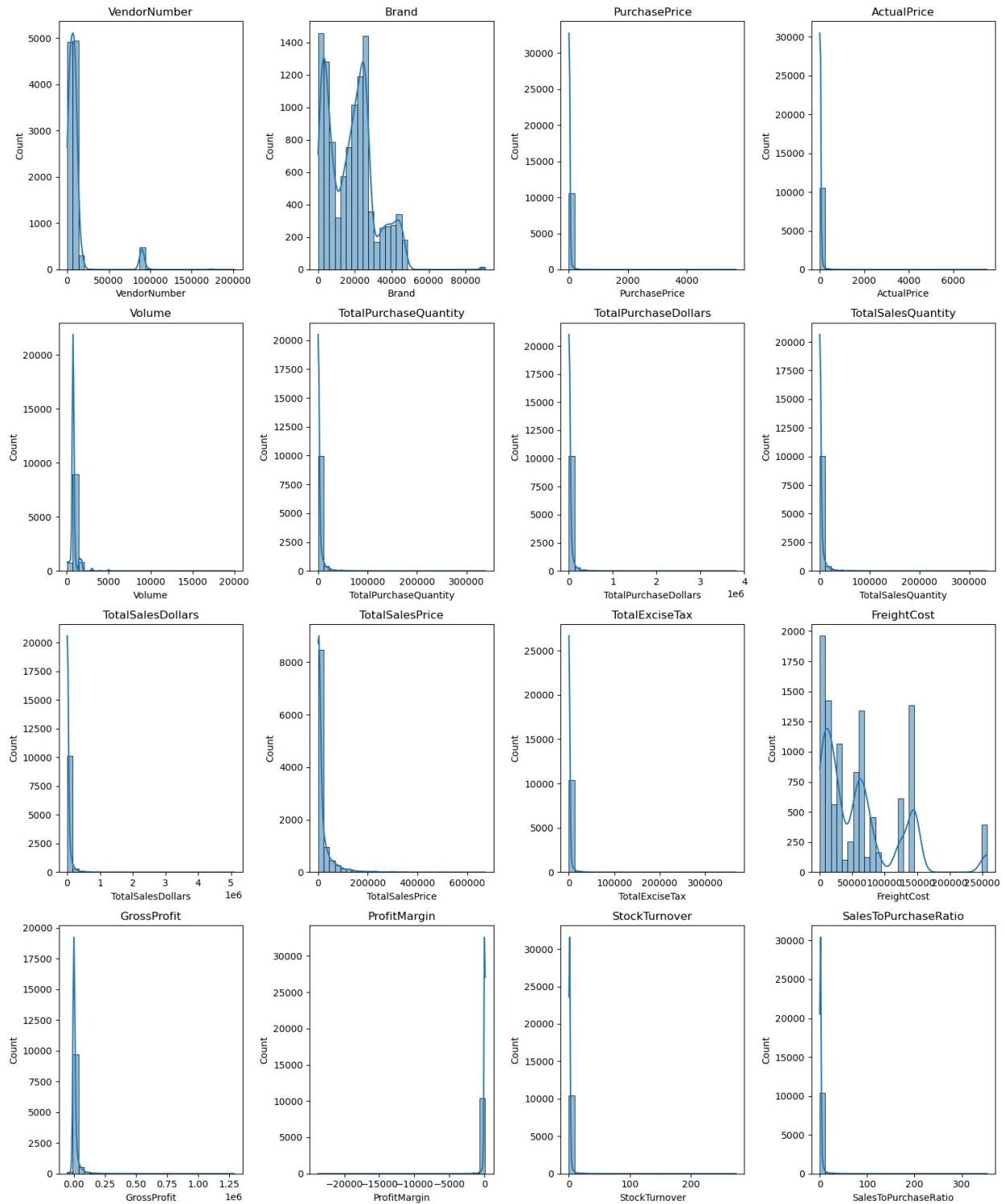
```
numerical_cols = df.select_dtypes(include=np.number).columns
```

```
plt.figure(figsize=(15,18))
```

```
for i, col in enumerate(numerical_cols):
```

```
plt.subplot(4,4,i+1)
sns.histplot(df[col],kde=True,bins=30)
plt.title(col)
plt.tight_layout()
plt.show

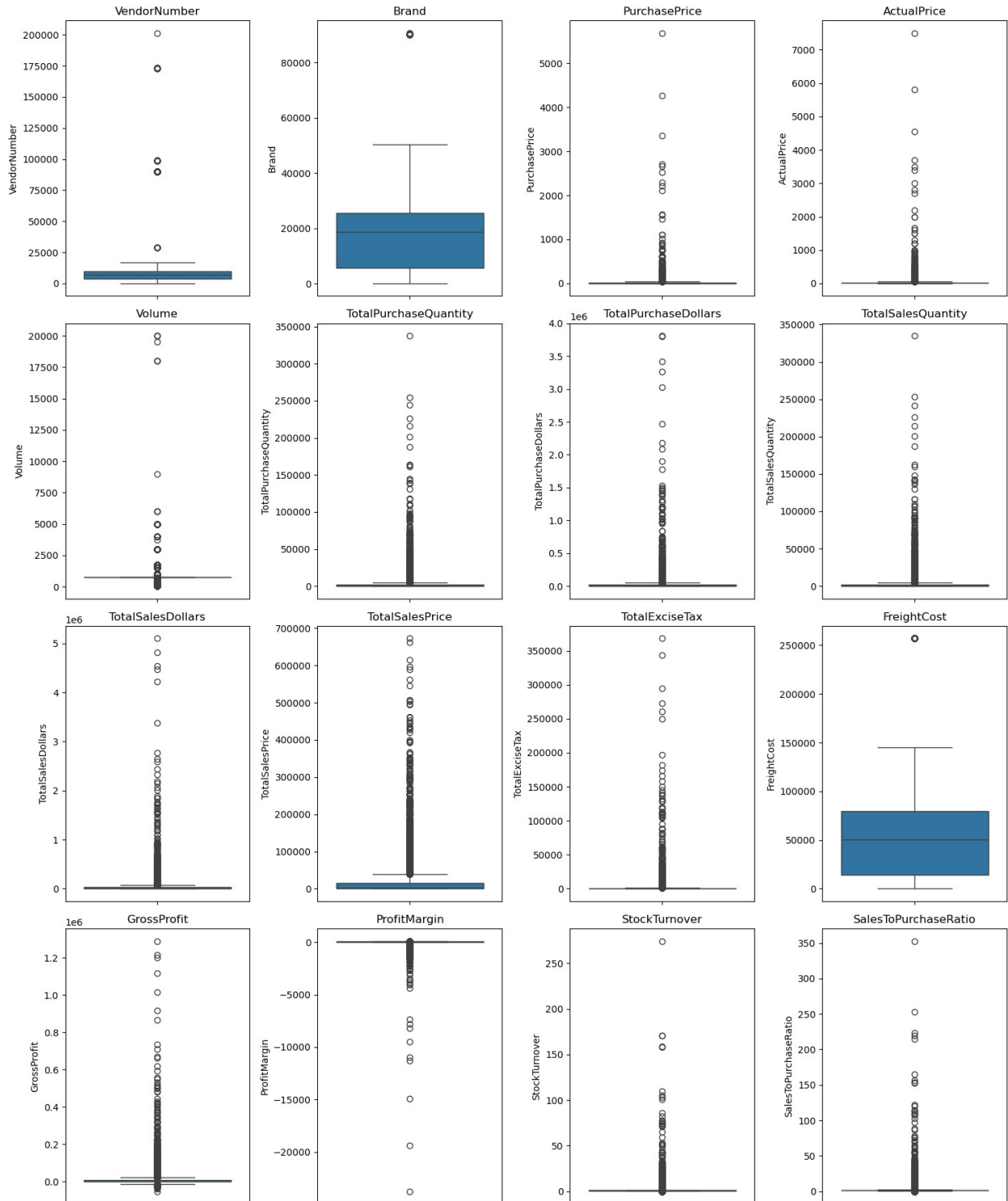
<function matplotlib.pyplot.show(close=None, block=None)>
```



```
# outliers detection with boxplot
plt.figure(figsize=(15,18))
for i, col in enumerate(numerical_cols):
    plt.subplot(4,4,i+1)
    sns.boxplot(y=df[col])
```

```
plt.title(col)
plt.tight_layout()
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



```
# filter the data by removing inconsistencies
```

```
df = pd.read_sql_query("""SELECT *
FROM vendor_sales_summary
WHERE GrossProfit > 0
AND ProfitMargin > 0
AND TotalSalesQuantity > 0""",conn)
```

```
df
```

	VendorNumber	VendorName	Brand \
0	1128	BROWN-FORMAN CORP	1233
1	4425	MARTIGNETTI COMPANIES	3405
2	17035	PERNOD RICARD USA	8068
3	3960	DIAGEO NORTH AMERICA INC	4261
4	3960	DIAGEO NORTH AMERICA INC	3545
...	...	...	...
8559	9815	WINE GROUP INC	8527
8560	8004	SAZERAC CO INC	5683
8561	3924	HEAVEN HILL DISTILLERIES	9123
8562	3960	DIAGEO NORTH AMERICA INC	6127
8563	7245	PROXIMO SPIRITS INC.	3065

	Description	PurchasePrice	ActualPrice	Volume
\				
0	Jack Daniels No 7 Black	26.27	36.99	1750.0
1	Tito's Handmade Vodka	23.19	28.99	1750.0
2	Absolut 80 Proof	18.24	24.99	1750.0
3	Capt Morgan Spiced Rum	16.17	22.99	1750.0
4	Ketel One Vodka	21.89	29.99	1750.0
...	...	...	...	...
8559	Concannon Glen Ellen Wh Zin	1.32	4.99	750.0
8560	Dr McGillicuddy's Apple Pie	0.39	0.49	50.0
8561	Deep Eddy Vodka	0.74	0.99	50.0
8562	The Club Strawbry Margarita	1.47	1.99	200.0
8563	Three Olives Grape Vodka	0.71	0.99	50.0

	TotalPurchaseQuantity	TotalPurchaseDollars	TotalSalesQuantity
\			
0	145080	3811251.60	142049.0

1	164038	3804041.22	160247.0
2	187407	3418303.68	187140.0
3	201682	3261197.94	200412.0
4	138109	3023206.01	135838.0
...	...	...	...
8559	2	2.64	5.0
8560	6	2.34	134.0
8561	2	1.48	2.0
8562	1	1.47	72.0
8563	1	0.71	86.0
TotalSalesDollars TotalSalesPrice TotalExciseTax FreightCost			
\			
0	5101919.51	672819.31	260999.20 68601.68
1	4819073.49	561512.37	294438.66 144929.24
2	4538120.60	461140.15	343854.07 123780.22
3	4475972.88	420050.01	368242.80 257032.07
4	4223107.62	545778.28	249587.83 257032.07
...	...	...	...
8559	15.95	10.96	0.55 27100.41
8560	65.66	1.47	7.04 50293.62
8561	1.98	0.99	0.10 14069.87
8562	143.28	77.61	15.12 257032.07
8563	85.14	33.66	4.46 38994.78
GrossProfit ProfitMargin StockTurnover SalesToPurchaseRatio			
0	1290667.91	25.297693	0.979108 1.338647
1	1015032.27	21.062810	0.976890 1.266830
2	1119816.92	24.675786	0.998575 1.327594
3	1214774.94	27.139908	0.993703 1.372493

4	1199901.61	28.412764	0.983556	1.396897
...	...	...	...	...
8559	13.31	83.448276	2.500000	6.041667
8560	63.32	96.436186	22.333333	28.059829
8561	0.50	25.252525	1.000000	1.337838
8562	141.81	98.974037	72.000000	97.469388
8563	84.43	99.166079	86.000000	119.915493

[8564 rows x 18 columns]

Distribution plots for numerical columns

numerical_cols = df.select_dtypes(include=np.number).columns

plt.figure(figsize=(15,18))

for i, col in enumerate(numerical_cols):

plt.subplot(4,4,i+1)

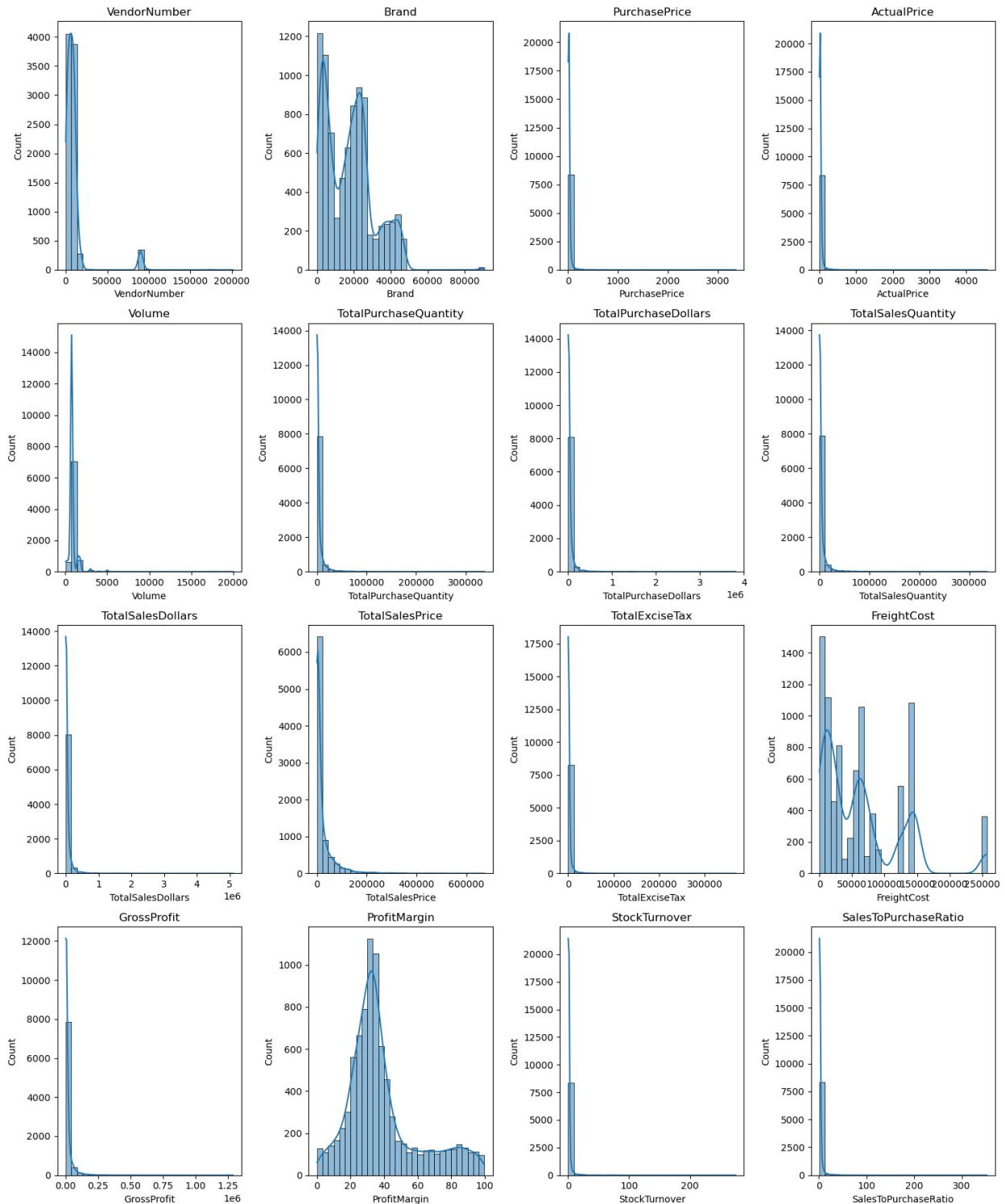
sns.histplot(df[col],kde=True,bins=30)

plt.title(col)

plt.tight_layout()

plt.show

<function matplotlib.pyplot.show(close=None, block=None)>

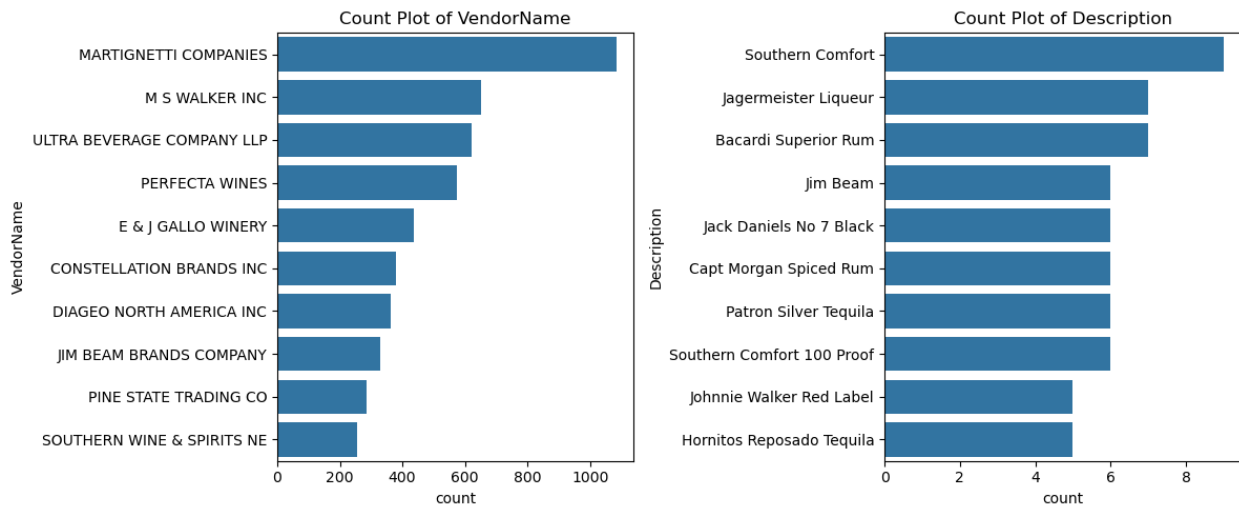


```
# count plot for categorical columns
categorical_cols = ["VendorName", "Description"]
```

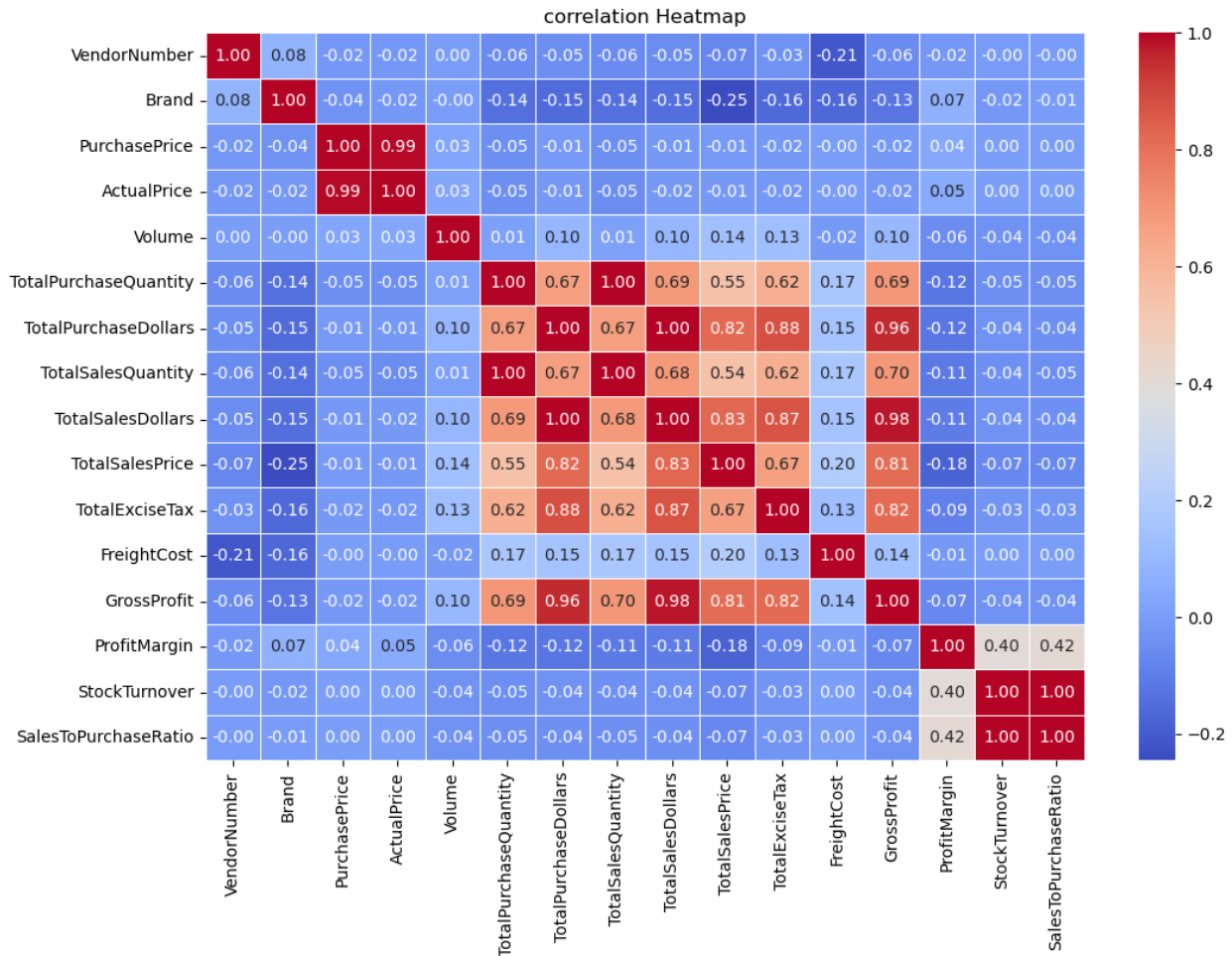
```
plt.figure(figsize=(12,5))
for i, col in enumerate(categorical_cols):
```

```
plt.subplot(1,2,i+1)
sns.countplot(y=df[col],order=df[col].value_counts().index[:10])
plt.title(f"Count Plot of {col}")
plt.tight_layout()
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



```
# Correlation Heatmap
plt.figure(figsize=(12,8))
correlation_matrix = df[numerical_cols].corr()
sns.heatmap(correlation_matrix, annot=True, fmt=".2f",
cmap="coolwarm", linewidth=0.5)
plt.title("correlation Heatmap")
plt.show()
```



Data Analysis

1. Identify Brands that need Promotional or Pricing Adjustments which Exhibit lower sales performance but higher profit margin

```
brand_performance = df.groupby("Description").agg({
    "TotalSalesDollars": "sum",
    "ProfitMargin": "mean" }).reset_index()

low_sales_threshold =
brand_performance["TotalSalesDollars"].quantile(0.15)
high_margin_threshold = brand_performance[
    "ProfitMargin"].quantile(0.85)

low_sales_threshold
np.float64(560.299)
high_margin_threshold
```

```
np.float64(64.97017552750113)
```

```
# Filter brands with low sales but high profit margins
```

```
target_brands = brand_performance[  
    (brand_performance["TotalSalesDollars"] <= low_sales_threshold) &  
    (brand_performance["ProfitMargin"] >= high_margin_threshold)  
]
```

```
print("Brands With low Sales but High Profit Margins")
```

```
display(target_brands.sort_values("TotalSalesDollars"))
```

Brands With low Sales but High Profit Margins

	Description	TotalSalesDollars	ProfitMargin
6199	Santa Rita Organic Svgn Bl	9.99	66.466466
2369	Debauchery Pnt Nr	11.58	65.975820
2070	Concannon Glen Ellen Wh Zin	15.95	83.448276
2188	Crown Royal Apple	27.86	89.806174
6237	Sauza Sprklg Wild Berry Marg	27.96	82.153076
...	...	...	...
5074	Nanbu Bijin Southern Beauty	535.68	76.747312
2271	Dad's Hat Rye Whiskey	538.89	81.851584
57	A Bichot Clos Marechaudes	539.94	67.740860
6245	Sbragia Home Ranch Merlot	549.75	66.444748
3326	Goulee Cos d'Estournel 10	558.87	69.434752

```
[198 rows x 3 columns]
```

```
brand_performance =
```

```
brand_performance[brand_performance["TotalSalesDollars"]<10000]
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(data=brand_performance,x='TotalSalesDollars',y='Profit  
Margin',color='blue',label='All Brands',alpha=0.2)
```

```
sns.scatterplot(data=target_brands,x='TotalSalesDollars',y='ProfitMarg  
in',color='red',label='Target Brands')
```

```
plt.axhline(high_margin_threshold,linestyle='--',color='black',label='  
High Margin Threshold')
```

```
plt.axvline(low_sales_threshold,linestyle='--',color='black',label='Lo  
w Sales Threshold')
```

```
plt.xlabel("Total Sales ($)")
```

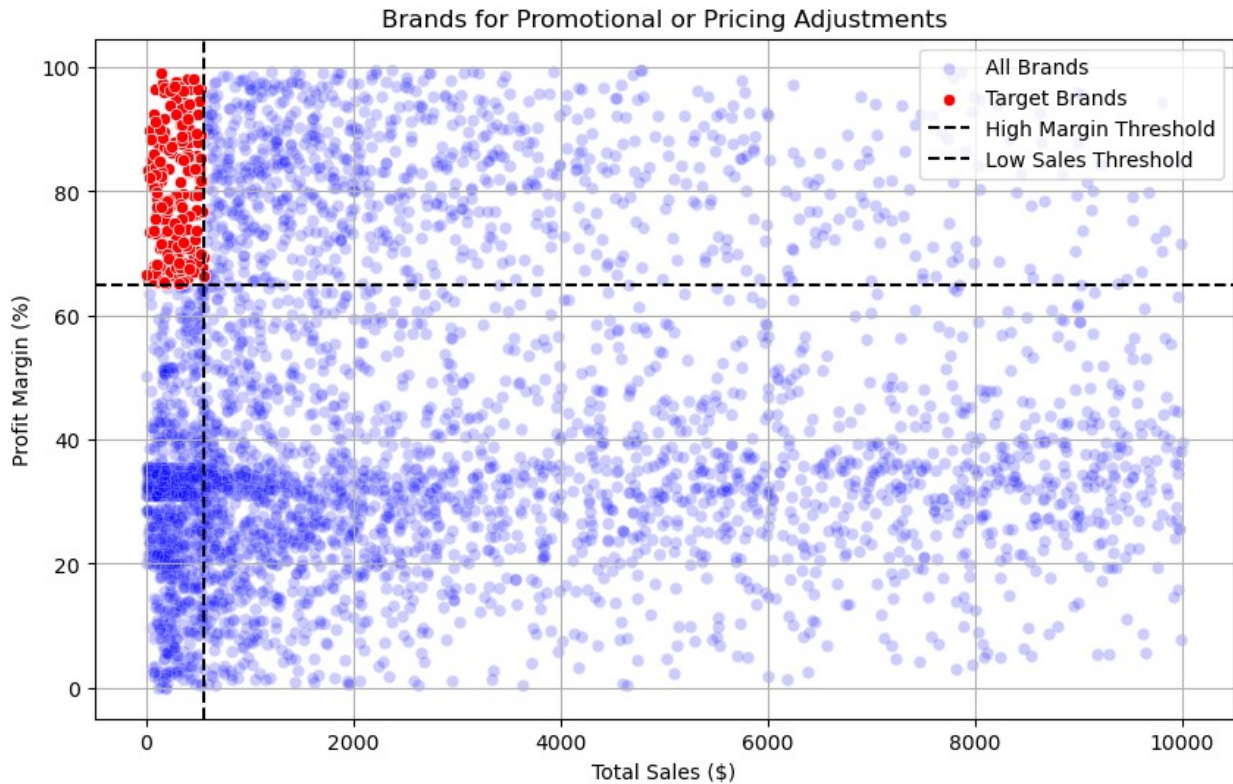
```
plt.ylabel("Profit Margin (%)")
```

```
plt.title("Brands for Promotional or Pricing Adjustments")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



2. Which vendors and brands demonstrate highest sales performance

Top Vendors and Brands by Sales Performance

```
top_vendors = df.groupby("VendorName")
["TotalSalesDollars"].sum().nlargest(10)
top_brands = df.groupby("Description")
["TotalSalesDollars"].sum().nlargest(10)
```

top_vendors

VendorName	TotalSalesDollars
DIAGEO NORTH AMERICA INC	67990099.42
MARTIGNETTI COMPANIES	39330359.36
PERNOD RICARD USA	32063196.19
JIM BEAM BRANDS COMPANY	31423020.46
BACARDI USA INC	24854817.14
CONSTELLATION BRANDS INC	24218745.65
E & J GALLO WINERY	18399899.46
BROWN-FORMAN CORP	18247230.65
ULTRA BEVERAGE COMPANY LLP	16502544.31
M S WALKER INC	14706458.51

Name: TotalSalesDollars, dtype: float64

top_brands

Description	TotalSalesDollars
Jack Daniels No 7 Black	7964746.76

Tito's Handmade Vodka	7399657.58
Grey Goose Vodka	7209608.06
Capt Morgan Spiced Rum	6356320.62
Absolut 80 Proof	6244752.03
Jameson Irish Whiskey	5715759.69
Ketel One Vodka	5070083.56
Baileys Irish Cream	4150122.07
Kahlua	3604858.66
Tanqueray	3456697.90

Name: TotalSalesDollars, dtype: float64

```
plt.figure(figsize=(15, 5))
```

```
# Plot for Top Vendors
```

```
plt.subplot(1, 2, 1)
```

```
ax1 =
```

```
sns.barplot(y=top_vendors.index,x=top_vendors.values,palette="Blues_r")
```

```
plt.title("Top 10 Vendors by Sales")
```

```
for bar in ax1.patches:
```

```
    ax1.text(
```

```
        bar.get_width() + (bar.get_width() * 0.02),
```

```
        bar.get_y() + bar.get_height() / 2,
```

```
        f'{bar.get_width():.0f}', # Added the text content (s  
parameter)
```

```
        ha='left',
```

```
        va='center',
```

```
        fontsize=10,
```

```
        color='black'
```

```
    )
```

```
# Plot for Top Brands
```

```
plt.subplot(1, 2, 2)
```

```
ax2 =
```

```
sns.barplot(y=top_brands.index.astype(str),x=top_brands.values,palette  
="Reds_r")
```

```
plt.title("Top 10 Brands by Sales")
```

```
for bar in ax2.patches:
```

```
    ax2.text(
```

```
        bar.get_width() + (bar.get_width() * 0.02),
```

```
        bar.get_y() + bar.get_height() / 2,
```

```
        f'{bar.get_width():.0f}', # Added the text content (s  
parameter)
```

```
        ha='left',
```

```

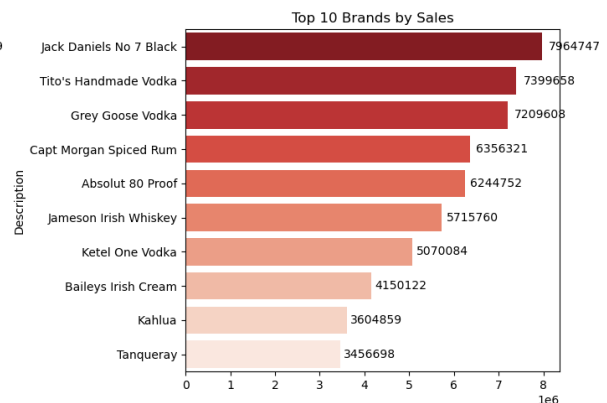
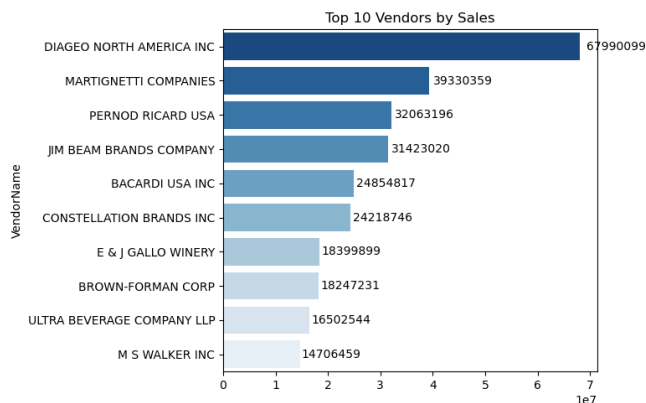
        va='center',
        fontsize=10,
        color='black'
    )

```

```

plt.tight_layout()
plt.show()

```



3. Which vendor contributes the most to total purchase dollars

```

vendor_performance = df.groupby("VendorName").agg({
    "TotalPurchaseDollars": "sum",
    "ProfitMargin": "sum",
    "TotalSalesDollars": "sum",
}).reset_index()

vendor_performance["Purchase_Contribution%"] =
vendor_performance["TotalPurchaseDollars"] / vendor_performance["TotalPurchaseDollars"].sum() * 100

vendor_performance =
round(vendor_performance.sort_values("Purchase_Contribution%", ascending=False), 2)

top_vendors = vendor_performance.head(10)

top_vendors["Cumulative_Contribution%"] =
top_vendors["Purchase_Contribution%"].cumsum()

top_vendors

```

	VendorName	TotalPurchaseDollars	ProfitMargin \
25	DIAGEO NORTH AMERICA INC	50097226.16	11950.63
57	MARTIGNETTI COMPANIES	25502095.83	47267.17
68	PERNOD RICARD USA	23851164.17	7580.51
46	JIM BEAM BRANDS COMPANY	23494304.32	11196.58
6	BACARDI USA INC	17432020.26	6074.52
20	CONSTELLATION BRANDS INC	15273708.08	14314.83

11	BROWN-FORMAN CORP	13238939.18	3800.58
30	E & J GALLO WINERY	12068539.22	14322.08
106	ULTRA BEVERAGE COMPANY LLP	11167081.61	25566.11
53	M S WALKER INC	9764312.60	26436.42

	TotalSalesDollars	Purchase_Contribution%
Cumulative_Contribution%		
25	67990099.42	16.30
16.30		
57	39330359.36	8.30
24.60		
68	32063196.19	7.76
32.36		
46	31423020.46	7.64
40.00		
6	24854817.14	5.67
45.67		
20	24218745.65	4.97
50.64		
11	18247230.65	4.31
54.95		
30	18399899.46	3.93
58.88		
106	16502544.31	3.63
62.51		
53	14706458.51	3.18
65.69		

```
fig, ax1 = plt.subplots(figsize=(10, 6))

sns.barplot(
    x='VendorName',
    y='Purchase_Contribution%',
    data=top_vendors,
    palette='mako',
    ax=ax1
)

for i, value in enumerate(top_vendors['Purchase_Contribution%']):
    ax1.text(i, value-1, f'{value:.1f}%',
             ha='center', fontsize=10, color='white')

ax2 = ax1.twinx()

ax2.plot(
    top_vendors['VendorName'],
    top_vendors['Cumulative_Contribution%'],
    color='red',
    marker='o',
    linestyle='dashed',
```

```

    label='Cumulative Contribution'
)

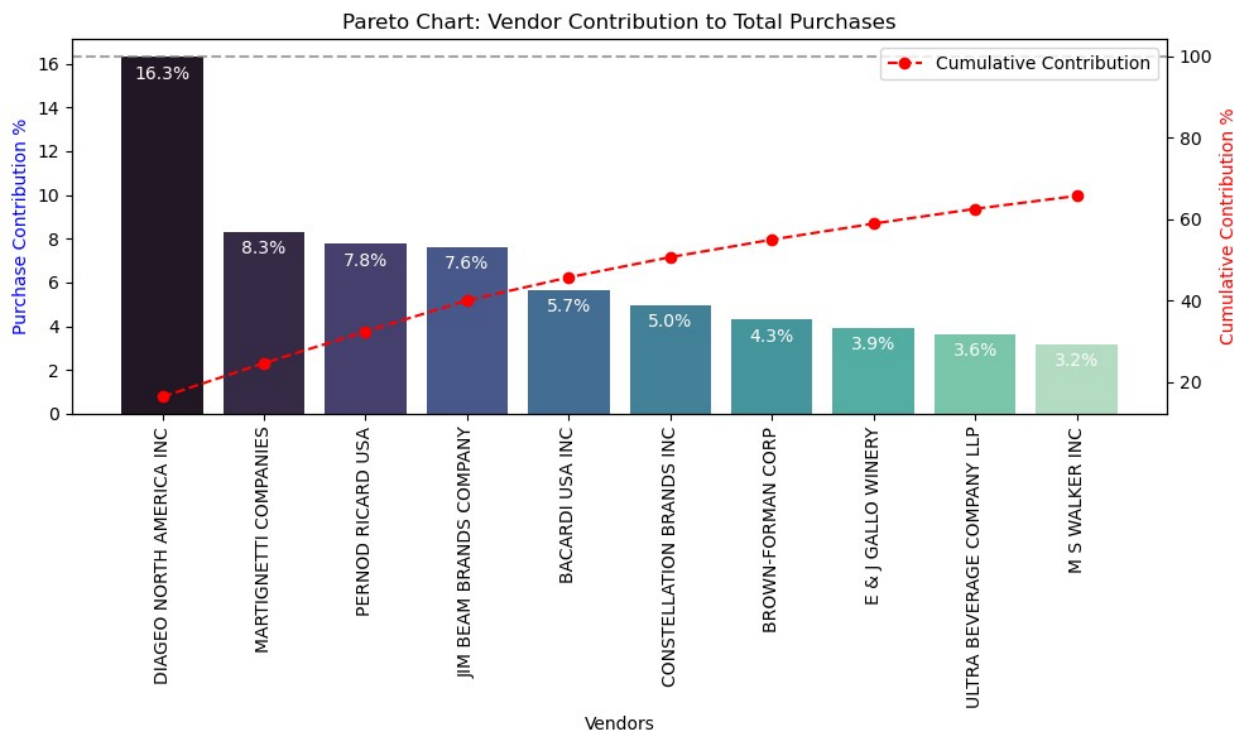
ax1.tick_params(axis='x', rotation=90)

ax1.set_ylabel('Purchase Contribution %', color='blue')
ax2.set_ylabel('Cumulative Contribution %', color='red')
ax1.set_xlabel('Vendors')
ax1.set_title('Pareto Chart: Vendor Contribution to Total Purchases')

ax2.axhline(y=100, color='gray', linestyle='dashed', alpha=0.7)
ax2.legend(loc='upper right')

plt.tight_layout()
plt.show()

```



4. How much of total procurement is dependent on the top vendors

```

print(f"Total Purchase Contribution of top 10 vendors is
{round(top_vendors['Purchase_Contribution%'].sum(),2)}%")

Total Purchase Contribution of top 10 vendors is 65.69%

vendors = list(top_vendors['VendorName'])
purchase_contributions = list(top_vendors['Purchase_Contribution%'])

total_contribution = sum(purchase_contributions)
remaining_contribution = max(0, 100 - total_contribution)

```

```

vendors.append("Other Vendors")
purchase_contributions.append(remaining_contribution)

fig, ax = plt.subplots(figsize=(8, 8))

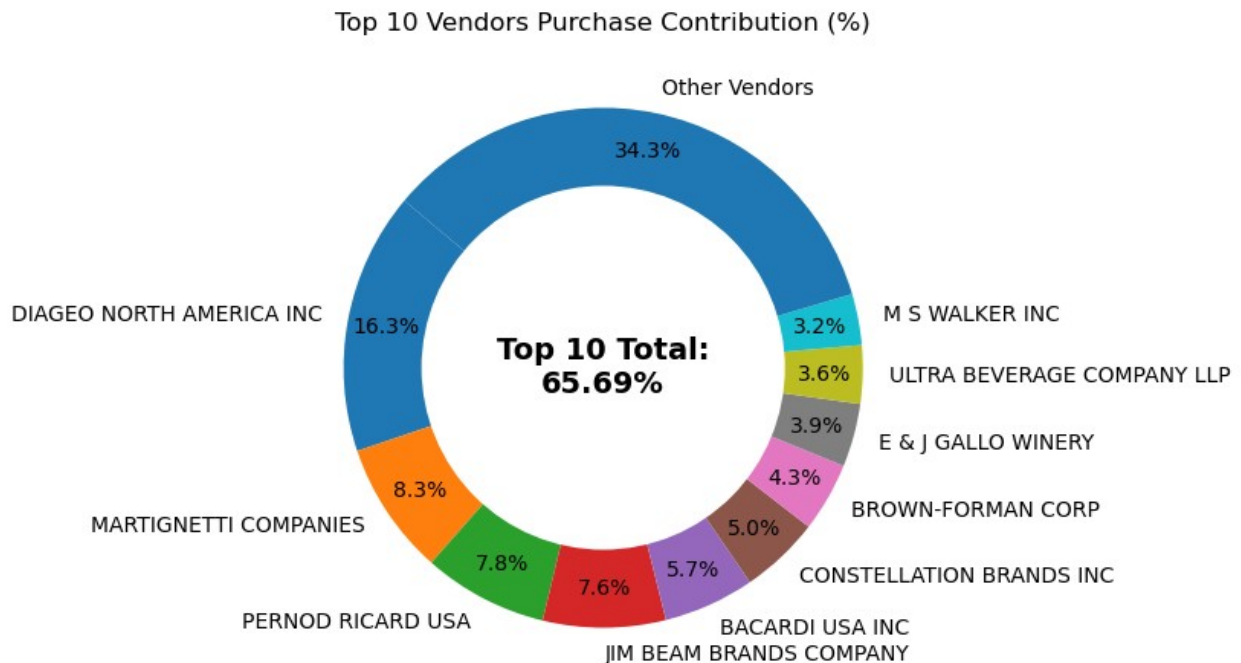
wedges, texts, autotexts = ax.pie(
    purchase_contributions,
    labels=vendors,
    autopct='%1.1f%%',
    startangle=140,
    pctdistance=0.85
)

centre_circle = plt.Circle((0, 0), 0.70, fc='white')
fig.gca().add_artist(centre_circle)

plt.text(
    0, 0,
    f"Top 10 Total:\n{total_contribution:.2f}%",
    fontsize=14,
    fontweight='bold',
    ha='center',
    va='center'
)

plt.title("Top 10 Vendors Purchase Contribution (%)")
plt.tight_layout()
plt.show()

```



5. Does purchasing in bulk reduce the unit price, and what is the optional purchase volume for cost savings?

```
df["UnitPurchasePrice"] =
df["TotalPurchaseDollars"]/df["TotalPurchaseQuantity"]

df["OrderSize"] =
pd.qcut(df["TotalPurchaseDollars"],q=3,labels=("Small","Medium","Large"))

df.groupby("OrderSize")[["UnitPurchasePrice"]].mean()
```

OrderSize	UnitPurchasePrice
Small	18.962158
Medium	30.002908
Large	16.380504

6. Which vendor has low inventory turnover, indicating excess stock and slow-moving products?

```
df[df["StockTurnover"]<1].groupby("VendorName")
[["StockTurnover"]].mean().sort_values("StockTurnover",ascending=True)
.head(10)
```

VendorName	StockTurnover
ALISA CARR BEVERAGES	0.615385

HIGHLAND WINE MERCHANTS LLC	0.708333
PARK STREET IMPORTS LLC	0.751306
Circa Wines	0.755676
Dunn Wine Brokers	0.766022
CENTEUR IMPORTS LLC	0.773953
SMOKY QUARTZ DISTILLERY LLC	0.783835
TAMWORTH DISTILLING	0.797078
THE IMPORTED GRAPE LLC	0.807569
WALPOLE MTN VIEW WINERY	0.820548

7. What is the confidence interval for profit margins of to- performing and low-performing vendors

```

top_threshold = df["TotalSalesDollars"].quantile(0.75)
low_threshold = df["TotalSalesDollars"].quantile(0.25)

top_vendors = df[df["TotalSalesDollars"] >= top_threshold]
["ProfitMargin"].dropna()
low_vendors = df[df["TotalSalesDollars"] <= low_threshold]
["ProfitMargin"].dropna()

top_vendors

0      25.297693
1      21.062810
2      24.675786
3      27.139908
4      28.412764
...
3523    79.684817
3681    85.782102
4751    93.085860
4920    95.012530
5050    94.271857
Name: ProfitMargin, Length: 2141, dtype: float64

def confidence_interval(data, confidence=0.95):
    mean_val = np.mean(data)
    std_err = np.std(data, ddof=1) / np.sqrt(len(data))
    t_critical = stats.t.ppf((1 + confidence) / 2, df=len(data) - 1)
    margin_of_error = t_critical * std_err
    return mean_val, mean_val - margin_of_error, mean_val +
margin_of_error

# Calculate Confidence Intervals
top_mean, top_lower, top_upper = confidence_interval(top_vendors)
low_mean, low_lower, low_upper = confidence_interval(low_vendors)

# Print Results
print(f"Top Vendors 95% CI: ({top_lower:.2f}, {top_upper:.2f}), Mean:

```

```

{top_mean:.2f}")
print(f"Low Vendors 95% CI: ({low_lower:.2f}, {low_upper:.2f}), Mean:
{low_mean:.2f}")

plt.figure(figsize=(12, 6))

# Top Vendors Plot
sns.histplot(top_vendors, kde=True, color="blue", bins=30, alpha=0.5,
label="Top Vendors")
plt.axvline(top_lower, color="blue", linestyle="--", label=f"Top
Lower: {top_lower:.2f}")
plt.axvline(top_upper, color="blue", linestyle="--", label=f"Top
Upper: {top_upper:.2f}")
plt.axvline(top_mean, color="blue", linestyle="-", label=f"Top Mean:
{top_mean:.2f}")

# Low Vendors Plot
sns.histplot(low_vendors, kde=True, color="red", bins=30, alpha=0.5,
label="Low Vendors")
plt.axvline(low_lower, color="red", linestyle="--", label=f"Low Lower:
{low_lower:.2f}")
plt.axvline(low_upper, color="red", linestyle="--", label=f"Low Upper:
{low_upper:.2f}")
plt.axvline(low_mean, color="red", linestyle="-", label=f"Low Mean:
{low_mean:.2f}")

plt.title("Confidence Interval Comparison: Top vs. Low Vendors")
plt.xlabel("Profit Margin (%)")
plt.ylabel("Frequency")
plt.legend()
plt.grid(True)
plt.show()

Top Vendors 95% CI: (30.74, 31.61), Mean: 31.18
Low Vendors 95% CI: (40.50, 42.64), Mean: 41.57

```

